F1 · DAY2 · WEEK3 SEMAINE 3 · JOUR 2 · FICHE 1 PARCOURS AI/ML ENGINEER

LUNDI 26 OCTOBRE 2026

Ton premier modèle de Machine Learning avec scikit-learn

Fabriquer des exemples X et y, entraîner une régression linéaire, la tester honnêtement sur des années jamais vues, puis prédire le bac 2027 en physique-chimie, maths et SVT. Chaque notion est énoncée, expliquée « en clair », puis appliquée à ExamPredict AI.

1. Du vote au modèle
2. scikit-learn, X et y
3. Fabriquer les exemples
4. La régression linéaire
5. Train / test dans le temps
6. Évaluer : l'erreur et le pif
7. Prédire 2027
8. Une fonction pour 3 matières
9. Les résultats
10. Données et vérification
11. Jour 1 / jour 2 : quand les stats battent le ML
12. Exercices corrigés
13. À savoir par cœur

---

1. Du vote au modèle

Au Day 1, la règle était écrite à la main : « l'année d'avant pèse 1, celle d'avant 0,8… ». Le 0,8 était choisi au feeling.

> **Définition 1.1** *(apprentissage automatique)*
>
> Un modèle de **Machine Learning** trouve lui-même sa règle à partir d'**exemples** : on lui montre des situations passées et leur résultat, il ajuste ses **paramètres** pour reproduire ces résultats au mieux.
>
> **En clair :** on ne lui donne pas la formule, on lui donne des exemples et il **devine la formule**.

2. scikit-learn, X et y

scikit-learn (saï-kit-leurne) est la bibliothèque de ML la plus utilisée en Python. Installation, une seule fois :

```
py -m pip install scikit-learn
```

NOM	RÔLE	DANS EXAMPREDICT
X (grand ixe)	les **entrées** (features)	combien de fois le chapitre est tombé les 3 années d'avant
y (petit i-grec)	la **cible** à deviner	combien de fois il tombe l'année suivante

3. Fabriquer les exemples

```
compte = pd.crosstab(df["annee"], df["famille"])   # tableau croisé : années × chapitres (cross-tab)
annees = list(compte.index)
lignes = []
for i in range(3, len(annees)):                     # à partir de la 4e année
    for famille in compte.columns:
        lignes.append({
            "annee": annees[i],
            "avant_1": compte.loc[annees[i - 1], famille],   # .loc = lire une case (point lok)
            "avant_2": compte.loc[annees[i - 2], famille],
            "avant_3": compte.loc[annees[i - 3], famille],
            "y": compte.loc[annees[i], famille],
        })
data = pd.DataFrame(lignes)
```

Définition 3.1 *(feature engineering)*

Transformer des données brutes en colonnes utiles pour le modèle s'appelle le **feature engineering (fi-tcheur èn-dji-ni-ringue)**. Ici, les « 3 années d'avant » sont des features que nous avons inventées.

En clair : on prépare les ingrédients avant de cuisiner. Un bon modèle avec de mauvais ingrédients donne un mauvais plat.

4. La régression linéaire

Définition 4.1 *(régression linéaire)*

Une **régression linéaire (ré-gré-sion li-né-èr)** prédit y par une somme pondérée des entrées :

$$y \approx a \times avant_1 + b \times avant_2 + c \times avant_3 + d$$

L'entraînement choisit a, b, c, d qui rendent les erreurs sur les exemples les plus petites possible.

En clair : le modèle cherche **combien chaque année passée doit compter**.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()          # un modèle vide
model.fit(X[train], y[train])       # LA ligne du ML : il apprend  (fite)
model.coef_                         # les poids a, b, c trouvés  (ko-èf)
model.predict(X[test])              # il devine  (pri-dikte)
```

5. Train / test dans le temps

Définition 5.1

On sépare les exemples en deux : le **train (tréïne)** pour apprendre, le **test (tèsste)** pour vérifier sur des données que le modèle n'a **jamais vues**.

```
train = data["annee"] < 2024     # apprendre sur le passé
test  = data["annee"] >= 2024    # examen sur 2024, 2025, 2026
```

Piège 5.2 *(couper au hasard)*

Quand les données suivent le **temps**, on coupe **dans le temps**, jamais au hasard. Sinon le modèle voit 2025 pour prédire 2024 : c'est de la triche, et son score est faux.

6. Évaluer : l'erreur et le pif

Définition 6.1 *(erreur absolue moyenne, MAE)*

La **MAE (mîne ab-so-lioute éreur)** est la moyenne des écarts entre la vraie valeur et la prédiction. MAE = 0,5 : le modèle se trompe en moyenne d'un demi-exercice.

Règle 6.2 *(toujours comparer au pif)*

Un modèle n'est utile que s'il bat une **référence simple** (baseline) : ici, deviner toujours la moyenne. Erreur du modèle < erreur au pif → le modèle apporte quelque chose.

```
from sklearn.metrics import mean_absolute_error
mean_absolute_error(y[test], pred)                              # erreur du modèle
mean_absolute_error(y[test], [y[train].mean()] * test.sum())    # erreur au pif
```

7. Prédire 2027

```
model.fit(X, y)                                  # réentraîner sur toutes les années
futur = pd.DataFrame({
    "avant_1": compte.loc[annees[-1]],           # -1 = dernière année (2026)
    "avant_2": compte.loc[annees[-2]],
    "avant_3": compte.loc[annees[-3]],
})
futur["prediction_2027"] = model.predict(futur).round(2)
```

Lecture : environ 1 ou plus → très probable ; environ 0,5 → une chance sur deux ; proche de 0 → peu probable.

8. Une fonction pour 3 matières

```
tout = pd.read_csv("data/chapitres_bac.csv")

def predire_matiere(matiere):
    df = tout[tout["matiere"] == matiere]        # filtrer une matière
    ...                                          # tout le modèle, décalé dans la fonction

for matiere in ["physique-chimie", "maths", "svt"]:
    predire_matiere(matiere)
```

> **Règle 8.1**
>
> On écrit le modèle **une fois** dans une fonction, et on le réutilise. Ajouter une matière revient à ajouter des lignes au CSV.

9. Les résultats

MATIÈRE	POIDS APPRIS	ERREUR MODÈLE	ERREUR PIF	TOP 2027
Physique-chimie	0,30 · 0,19 · 0,27	0,50	0,60	Mécanique (1,11), Chimie organique, Ondes et optique, Cinétique
Maths	0,51 · 0,14 · 0,06	**0,76**	1,22	Probabilités (2,73), Fonctions, Géométrie, Suites, Intégrales, Équations différentielles
SVT	0,29 · 0,17 · 0,12	0,59	0,64	Biologie végétale (0,97), Système nerveux, Climat

Dans le métier

Chaque matière a sa « personnalité » : en maths, l'année d'avant pèse 0,51, ce qui est tombé l'an dernier retombe. Et le modèle a découvert seul que les **nombres complexes** ne tombent plus (0,18) : ils ont été retirés du programme.

10. Données et vérification

Piège 10.1 *(sources contradictoires)*

La liste maths de ChatGPT disait « 2021 : épreuves annulées », alors que les listes physique et SVT donnaient des sujets 2021. Vérification faite : les épreuves de spécialité 2021 ont bien été **annulées** (Covid) et remplacées par le contrôle continu. Les lignes 2021 ont été supprimées.

Choix de nettoyage

Exercice à plusieurs thèmes (« Fonctions et intégrales ») → une ligne par chapitre. Exercice « au choix » → les deux thèmes comptent. Exercice « à vérifier » → ignoré.

11. Jour 1 / jour 2 : quand les stats battent le ML

Depuis la réforme, il y a deux sujets par an. Un modèle par jour, testé sur 2026, donne : physique 0,39 contre 0,42 au pif, SVT 0,47 contre 0,48, maths 0,35 contre 0,25 (pire que le pif).

Proposition 11.1

Avec trop peu d'exemples (5 années par jour), un modèle ne peut pas apprendre : de simples **statistiques** (moyenne, médiane, écart-type) font aussi bien ou mieux. Un ingénieur honnête utilise alors les stats.

Dans le métier

Le remède n'est pas un modèle plus compliqué, c'est **plus de données** : les autres sessions de chaque année (Centres étrangers, Amérique du Nord, Asie, Polynésie…) donnent 6 à 8 sujets par an au lieu d'un.

2027	JOUR 1	JOUR 2
Physique-chimie	Mécanique · Chimie organique · Ondes et optique	Mécanique · Chimie organique · Ondes et optique
Maths	Probabilités · Fonctions · Suites · Géométrie	Fonctions · Probabilités · Suites · Géométrie
SVT	Climat · Biologie végétale	Biologie végétale · Climat

Les numéros d'exercice suivent l'ordre de probabilité, pas l'ordre réel du sujet, qui ne suit aucune logique.

12. Exercices corrigés

Exercice 1 ★ X et y

Pour prédire la note d'un élève à partir de ses heures de révision et de sommeil, que valent X et y ?

Corrigé. X = les colonnes révision et sommeil (les entrées) ; y = la note (la cible).

Exercice 2 ★ lire une prédiction

Les poids appris sont [0,5 · 0,2 · 0,1], le bonus vaut 0,1. Un chapitre est tombé 2, 1 et 0 fois les 3 dernières années. Prédiction ?

Corrigé. 0,5 × 2 + 0,2 × 1 + 0,1 × 0 + 0,1 = 1,3 exercice attendu.

Exercice 3 ★★ train / test

Pourquoi ne pas mélanger les années au hasard avant de couper en train et test ?

Corrigé. Le modèle verrait des années futures pendant l'apprentissage. Son erreur de test serait trop optimiste : il aurait triché (Piège 5.2).

Exercice 4 ★★ baseline

Un modèle a une erreur de 0,9 ; deviner la moyenne donne 0,8. Que conclure ?

Corrigé. Le modèle est inutile : il fait pire que le pif. Il faut plus de données ou de meilleures features, ou garder la baseline.

Exercice 5 ★★★ données

Deux sources donnent des informations contradictoires sur l'année 2021. Que fait un ingénieur ML ?

Corrigé. Il vérifie avec une source fiable et indépendante, corrige le jeu de données, relance le modèle et note la correction (ici, dans le message de commit).

13. À savoir par cœur

1. Un modèle de ML apprend sa règle à partir d'exemples : `model.fit(X, y)`.
2. **X** = les entrées (features), **y** = la cible à deviner.
3. Le **feature engineering** prépare les colonnes utiles (ici : les 3 années d'avant).
4. La **régression linéaire** prédit une somme pondérée ; `model.coef_` donne les poids appris.
5. Données dans le temps → on coupe train / test **dans le temps**.
6. La **MAE** mesure l'erreur moyenne ; un modèle doit battre le pif.
7. Après l'examen, on réentraîne sur tout, puis `model.predict` sur le futur.
8. Une **fonction** rend le modèle réutilisable pour n'importe quelle matière.
9. On vérifie les données contradictoires (2021 annulé) avant de faire confiance au modèle.
10. Trop peu de données → les stats battent le ML ; le remède, c'est plus de données.
